

Exercise: Simple CNN Image Classifier (Intel Image Classification)

Dataset: [Intel Image Classification](#) — Kaggle, ~350MB, 6 balanced classes: buildings, forest, glacier, mountain, sea, street

Setup

```
pip install kaggle tensorflow scikit-learn matplotlib
```

```
mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/  
kaggle datasets download -d puneet6060/intel-image-classification  
unzip intel-image-classification.zip -d intel_data
```

You should end up with folders like:

```
intel_data/seg_train/seg_train/{buildings,forest,glacier,mountain,sea,street}/  
intel_data/seg_test/seg_test/{buildings,forest,glacier,mountain,sea,street}/
```

Learning objectives

- Build a basic image preprocessing + augmentation pipeline for a multi-class problem
 - Build and train a small CNN from scratch
 - Evaluate a multi-class classifier with a confusion matrix (not just overall accuracy)
-

Tasks

Task 1 — Look at the data

- Count images per class in the train folder — confirm they're roughly balanced.
- Display a few sample images per class. Note the image size and that they're RGB (3 channels), not grayscale like the X-ray exercise.

Task 2 — Load and preprocess

Use `tf.keras.utils.image_dataset_from_directory` to load train and test sets directly — it handles resizing and batching for you, so this step is much shorter than manual preprocessing:

- Resize every image to 100×100 (smaller than the original 150×150 — keeps training fast on a laptop)
- Normalize pixel values to $[0, 1]$
- Split off 15% of the training data as a validation set (the function has a built-in `validation_split` argument)

Then add a small augmentation step (`RandomFlip` , `RandomRotation` , `RandomZoom`) applied only to the training pipeline.

Task 3 — Build the CNN

Keep it small — this is a 6-class problem but the images are visually distinct, so you don't need a deep network:

- 2–3 Conv2D + MaxPool2D blocks
- One Dropout layer
- Final Dense layer with 6 outputs and `softmax` activation (not sigmoid — this is multi-class, not binary)

Task 4 — Train

- Compile with `sparse_categorical_crossentropy` loss (since labels are integers 0–5, not one-hot) and accuracy as the metric.
- Add early stopping on validation loss.
- Train for up to 15–20 epochs.

Task 5 — Evaluate

- Run predictions on the test set.
- Print a classification report (precision/recall/F1 per class) and a 6×6 confusion matrix.

- Look at which classes get confused with each other most (hint: glacier vs mountain, and sea vs glacier, are common mix-ups — see if your model makes the same mistakes and think about why those pairs are visually similar).

Bonus (optional)

- Try removing the augmentation step and compare validation accuracy — quantify whether it actually helped on this dataset.
 - Try increasing to 4 conv blocks and see if accuracy improves or if you start overfitting instead (watch the train/val loss gap).
-

What "done" looks like

- Final test accuracy and a 6×6 confusion matrix